

Proposition d'épreuves d'entraînement (CC3)

Voici une proposition d'épreuves d'entraînement originales pour un **CC3**, conçues en respectant le format et les exigences des sources (langage C obligatoire, robustesse, gestion dynamique de la mémoire et scripts Shell).

Exercice 1 : C – L'Arbre de Somme Harmonisé (7 pts)

Contexte : On considère un arbre binaire de recherche (ABR) particulier où chaque nœud doit respecter une règle de “Somme Partielle”. La structure utilisée est la suivante :

```
1 typedef struct _node {
2     int valeur;
3     int somme_descendants; // Somme de toutes les valeurs des fils gauche et droit
4     struct _node *fg, *fd;
5 } Noeud, *Arbre;
```

1. **Insertion Spécifique :** Écrire une fonction `Arbre insererSomme(Arbre a, int val)` qui insère une valeur dans l'ABR selon les règles classiques. Lors de l'insertion, la fonction doit mettre à jour le champ `somme_descendants` de tous les nœuds ancêtres traversés.
 2. **Vérification d'Harmonie :** Un arbre est dit “harmonisé” si, pour chaque nœud, sa valeur est strictement inférieure à sa `somme_descendants`. Écrire une fonction récursive `int estHarmonise(Arbre a)` qui retourne 1 si l'arbre est harmonisé, 0 sinon.
 3. **Rotation et Intégrité :** Si l'on effectue une **rotation simple à gauche** sur un nœud pour équilibrer l'arbre (type AVL), expliquez brièvement si le champ `somme_descendants` doit être recalculé pour le pivot et l'ancienne racine. Justifiez en vous appuyant sur le principe de rotation.
-

Exercice 2 : C – Gestion d'une File de Priorité de Colis (7 pts)

Contexte : Un centre logistique gère l'expédition de colis. Les colis arrivent dans une file (FIFO), mais certains colis “Prioritaires” doivent être traités avant les autres.

1. **Structures :** Déclarer une structure `Colis` (`id`, `poids`, `estPrioritaire`) et la structure `File` permettant de gérer une file dynamique de colis via des pointeurs.
2. **Enfilage Intelligent :** Écrire une procédure `enfilerColis(File* f, Colis c)` :
 - Si le colis n'est pas prioritaire, il est ajouté en fin de file.
 - S'il est prioritaire, il est inséré juste après le dernier colis prioritaire déjà présent dans la file, ou en tête si aucun ne l'est.

3. **Traitement et Libération** : Écrire une fonction `Colis* traiterColis(File* f)` qui retire le premier colis de la file, affiche ses informations et retourne son adresse. Attention : toute la mémoire doit être allouée dynamiquement et vous devez assurer la robustesse (vérifier si la file est vide).
-

Exercice 3 : Shell – Analyse de Logs de Connexion (6 pts)

Contexte : Vous disposez d'un fichier `connexions.log` dont chaque ligne suit le format : `date;identifiant;duree_secondes;statut(OK/FAIL)`

Exemple :

```
— 2023-10-01;user123;120;OK
— 2023-10-01;admin;0;FAIL
```

Écrire un script Shell qui prend le nom du fichier en argument et réalise les actions suivantes :

1. **Validation** : Vérifier que l'argument est bien un fichier existant et lisible. Sinon, afficher un message d'erreur sur **la sortie d'erreur standard** et quitter avec le code 1.
 2. **Filtrage** : Créer un fichier `echecs.txt` contenant uniquement les lignes dont le statut est FAIL.
 3. **Extraction** : Afficher la liste unique (sans doublons) de tous les identifiants ayant réussi à se connecter (OK).
 4. **Calcul de Performance** : Calculer et afficher la **durée totale** de connexion pour toutes les sessions réussies. *Aide : vous pouvez utiliser une boucle `for` pour parcourir les durées extraites avec `cut`.*
 5. **Alerte** : Si un identifiant apparaît plus de 3 fois avec le statut FAIL, afficher « Alerte : tentative d'intrusion pour l'utilisateur <id> ».
-

Conseils pour réussir (selon les sources)

- **Robustesse** : En C, vérifiez toujours les retours de `malloc` et les pointeurs NULL avant usage.
- **Modularité** : Pour les arbres, privilégiez les solutions récursives qui sont plus “compactes et élégantes”.
- **Shell** : N'oubliez pas les guillemets autour des variables et les espaces dans les crochets des tests [`$a -eq $b`].